

Developer experience report — SecondWave

Track 2 — Lending · XRPL public Devnet · September 2026

Who this is for: the judges and the Ripple team. **What it is:** what we actually went through building a product on the XRPL lending protocol — not a catalogue of error codes.

Team: Hugo (vaults, loans, resale) · Noé (risk analyst, wallet) **Track:** 2 — Lending · private vaults + on-chain credentials **Network:** XRPL public Devnet · September 2026

Technical evidence (scripts, hashes, versions) is in the appendix. Here we describe what blocks a human.

In thirty seconds

1. **A lender’s “collateral” is almost never what you think it is.** A default can push **95 % of the loss onto depositors**, even when the broker shows a large reserve.
2. **While the money is locked in, the displayed price can stay beautiful.** A late loan changes nothing on screen until someone formally records it. We found **22 vaults in exactly that state** among the 40 largest on the network, without having created a single one.
3. **“Success” does not mean the trade happened.** Delivery and no-op are indistinguishable in the response — we froze two Devnet transactions side by side to prove it.
4. **The error messages name neither the field nor the rule**, which turned two of our findings into lost afternoons.

What we built

SecondWave is an **exit market** for closed-ended vault shares, with a **risk rating** next to every offer.

The idea is simple. Once money is deposited in a fixed-term vault, it cannot be withdrawn until the redemption date. During that time, the loans inside can go bad. We let you **resell your share** to someone else, and we say whether the discount is an **opportunity** (someone needs cash) or a **trap** (the vault is already sick).

Getting there meant running the whole cycle on Devnet: credentials, vaults, deposits, brokers, loans, defaults, then resale. It is that cycle — not an isolated tutorial — that surfaced the frictions below.

1. “Collateral” is not what you think

This is **the** point of the project. Hugo measured it at the protocol level; Noé put it at the heart of the analyst.

We are told about **first-loss capital**: a reserve the broker deposits, which should be eaten **before** the savers’ money. The name — and often the screen — suggests: “if they have 5,000 in the pot, that 5,000 protects depositors.”

That is not what the network does.

At default, the protocol takes **only a small computed slice** — a percentage of a percentage. The rest of the pot **stays with the broker**. Depositors pay the difference. And they cannot leave the vault while it happens.

Same numbers, two readings:

What you see	What actually happens
Reserve shown: 5,000	Only 500 can be taken at default
“We’re well covered”	95 % of the loss goes to depositors
The surplus looks reassuring	The remaining 4,500 never move

The official documentation example **already computes** this kind of result (about 11 of protection against a 1,090 default). It never says it in one sentence: *almost none of that default is first loss*.

Two corollaries, just as simple:

- **As soon as no loan is outstanding**, the broker can **pull back 100 %** of the reserve. It is not a permanent cushion.
- **A late loan does not lower the vault’s price.** The displayed figure only moves when someone formally declares a loss. Until then, the depositor is stuck behind a price that is too good.

What we’re asking for: one sentence, clearly visible, on the “First-Loss Capital” page: *the real protection at default is not the money in the pot, it is the small cap set by the rates*. And, later, a pre-computed field (“maximum protection”) so wallets don’t have to reinvent it.

2. Once you’re in, you don’t get out — sometimes not at all

A fixed-term vault **forbids withdrawal** for the whole investment period. That is the product. We accepted it.

The trap is the **combination** with on-chain identity. Our vaults are private: only holders of an accepted credential can enter, and also **receive** a share.

If that credential expires or is revoked while the money is locked in:

- you **cannot withdraw** (too early);

- you **cannot sell** to an otherwise valid buyer (the network refuses the transfer).

The person who most needs an emergency exit is the one denied it. Withdrawal remains possible **at the redemption date**, even without a credential — but until then the position is frozen.

What we’re asking for: decide clearly whether selling *outward* (leaving the vault) should stay possible when you are no longer a member. Today the safety net arrives too late.

3. The official wallet cannot deposit XRP

Noé tried to subscribe from the **Ripple wallet extension**, as a normal user would, not as a script.

The vault expects an XRP amount in a plain form. The extension always sends the amount **as if it were a token** (an object with a currency). The network refuses. The message only says the field is invalid — **without saying the type is wrong**. The user **cannot edit** the JSON.

Result: **dead end**. You cannot join an XRP vault from the official tool.

We fixed the bug in a copy of the extension and opened a pull request. SecondWave itself is not at fault here: this is a depositor’s first real contact with XLS-65.

What we’re asking for: if the vault is in XRP, send the amount as a plain string; keep the “token” object for IOUs only. And a message that says *“this vault expects XRP, not a token”*.

4. “Success” does not mean the trade happened

This is the friction that cost us the most code, and the one we did not see coming. It concerns **XLS-56 (Batch)**, the transaction our whole settlement rests on.

To resell a share we send **several operations at once**, all or nothing: the buyer authorises themselves to receive the shares, the shares move, the money moves. Three legs, one transaction.

The network answers **success**. The problem is that this word does not separate two opposite situations.

What the server returns when the trade happened, and when it did nothing:

Field	Trade succeeded	Nothing applied
Response	<code>accepted</code>	<code>accepted</code>
Applied	<code>true</code>	<code>true</code>
Code	<code>tesSUCCESS</code>	<code>tesSUCCESS</code>
Text	“The transaction was applied”	“The transaction was applied”
Metadata	no value movement	no value movement

Eight sends replayed: five delivered, three did nothing. **One single observed combination**. Nothing in the response separates them.

We froze the demonstration on-chain so it can be checked without us. Two Devnet transactions, openable in the explorer: one delivered 3 XRP, the other did nothing. Here is everything the ledger records about each:

	A — delivered	E — did nothing
Code	<code>tesSUCCESS</code>	<code>tesSUCCESS</code>
Metadata nodes	Deleted Ticket, Modified AccountRoot, Modified DirectoryNode	Deleted Ticket, Modified AccountRoot, Modified DirectoryNode
Balance change	−150 drops (the fee)	−150 drops (the fee)

Rigorously identical. Neither leaves, in its own metadata, the faintest trace of what moved — or didn’t. The links are in probes-marche/PREUVES.md.

We are not saying the code is wrong. “Nothing” is a legitimate outcome of all-or-nothing. We are saying you cannot tell which case you are in. XLS-56 does specify a per-leg execution report, **exactly for this**. It is absent from what the server returns.

Three consequences we had to pay for in code:

- **Rebuild the legs by hand.** They exist in the ledger, as separate transactions in the same block, with their own hashes. You have to go and fetch them account by account over the block’s range. It is the most fragile part of our settlement, and it only reconstructs what the server already knows.
- **Compare balances before and after.** That is our only proof a trade happened. Any serious product will have to do the same.
- **Never pre-check.** The simulation tool refuses batched sends — it answers “not implemented”. The transaction everything depends on is the only one you cannot dry-run.

And the choice of mode is a silent trap. We replayed the same exchange under all four send modes. **Three out of four hand over the shares without collecting the price**, while announcing success. You cannot forget to pick a mode — but you can pick the wrong one, and nothing flags it. For value-against-shares, only one is correct. Two details from the same family, found by typing next to the target:

- a batched send with **fewer than two legs** is refused with “array is empty”, when the array is not;
- the buyer’s authorisation leg **must come first**. Placed anywhere else, the share delivery fails and everything is rolled back. Nothing says so.

What we’re asking for:

1. That the response carry **the index of the failing leg and its code**. One integer and one code — the field is already specified, it just needs filling.
 2. A warning at the top of the page: *to exchange value, use all-or-nothing only*, and a helper in the SDKs that enforces it.
 3. That the simulation tool accept batched sends. It is the transaction we most need to test.
-

5. Error messages don’t say what to fix

This was the running theme of the whole weekend. A few scenes from real life:

“Malformed transaction.” A vault refused because the dates are too close, too far apart, because a field is forbidden on XRP, or because a flag is missing. The server says the same sentence every time. The useful messages exist only **inside the JavaScript SDK**. Anyone signing another way gets nothing.

“Expired.” That single word means four different things: a repayment slightly too late, a deposit outside the entry window, a loan started too late, an expired credential. Impossible to tell which without re-reading everything by hand. And for the late repayment, the network expects a **special flag** the message never mentions.

“No permission.” Often it is not about rights at all. It is “your loan dates are too close to the vault’s redemption” or “this vault type does not accept a broker”. You go looking for an identity problem; it is a calendar problem.

An invisible rule, a lost session. A loan’s grace period must last **at least one minute**, and **no longer** than the interval between two payments. Nowhere in the docs. The refusal is, again, “malformed”. We tested three false leads before sweeping everything.

Same family: three fields are all called “rate” and do not mean the same thing (annual interest, reserve floor, share liquidated at default). Without a table you get the display wrong by a factor of 10 or 20.

What we’re asking for: name **the field** and **the rule** in the message. **expired** and **no permission** should become sentences like *“late repayment: add the corresponding flag”* or *“last payment too close to the redemption date”*.

In closing

Five findings, one shape. **The protocol is sound; what is missing is what it tells you.** The vault holds, all-or-nothing holds, the identity gate is watertight, rounding is exact over hundreds of round trips — we tried to break these and could not. What cost us days was never the ledger being wrong, it was the ledger being silent: a collateral figure that does not mean what it says, a price that does not move when the risk does, a success code covering two opposite outcomes, and refusals naming neither field nor rule.

Every one of those is a documentation or a reporting fix, not a protocol change. That is the good news.

Cut for length, each measured and reproducible in the repository:

- **No read method for a loan, a broker, the share holders — or the list of vaults.** We sweep 489 vaults in 11 s and re-read each one to find anything.
 - **Devnet is ahead of the docs** (three-phase vaults, date rules, share escrow). We learned it by typing next to the target.
 - **A signed offer did not survive waiting** — 72 s, and dead the moment either party transacted. Reserved transaction numbers fix it entirely; that is written nowhere, and it is the difference between an order book and an appointment.
 - **No wallet can sign a batched exchange**, so every peer-to-peer market is forced to be custodial.
 - **A proposal:** let an escrow point at a **permissioned domain** instead of a named destination. In a private vault everyone who could answer is already credentialled — naming the destination twice adds no safety, it only removes the possibility of a market.
-

Where the proofs are

- **probes-marche/PREUVES.md** — 8 Devnet transactions, openable in the explorer, including the A / E pair of section 4.
- **FEEDBACK.md**, **FEEDBACK_XLS65_XLS66.md** — long technical reports. **probes/FRICTIONS.md**, **probes-marche/FRICTIONS.md**, **FRICTIONS-annexe.md** — reproductions. **probes-marche/12...16** — the measurements behind the cut findings.

About 450 cases replayed on Devnet, thirty vaults created and 489 swept. Library **xrpl@5.2.0-beta.1**, rippled 3.4.0-rc5. *Security leads are passed on verbally, as the rules require.*